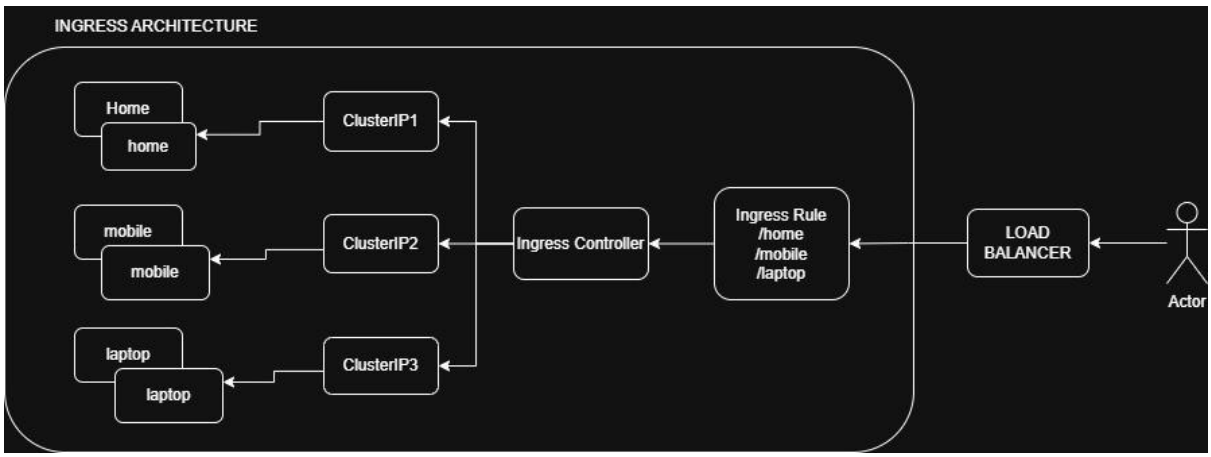
INGRESS AND LOAD BALANCING IN KUBERNETES

WHAT IS INGRESS

1. it can expose the http and https ports from outside the cluster to the service within the cluster
2. it works like a load balancer that generally distributes traffic to the evenly pods and their services.

TYPES OF INGRESS CONTROLLER

1. kong
2. istio
3. nginx



WRITING MANIFEST FOR THE NGINX-INGRESS

1. vim deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-home
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-home
  template:
    metadata:
```

```
      labels:
        app: nginx-home
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd-mobile
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpd-mobile
  template:
    metadata:
      labels:
        app: httpd-mobile
    spec:
      containers:
      - name: httpd
        image: httpd:latest
        ports:
        - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tomcat-laptop
spec:
  replicas: 1
  selector:
    matchLabels:
      app: tomcat-laptop
  template:
    metadata:
```

```
    labels:
      app: tomcat-laptop
  spec:
    containers:
    - name: tomcat
      image: tomcat:latest
      ports:
      - containerPort: 8080
```

2. **vim service.yaml**

```
apiVersion: v1
kind: Service
metadata:
  name: my-svc-home
spec:
  selector:
    app: nginx-home
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-svc-mobile
spec:
  selector:
    app: httpd-mobile
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-svc-laptop
spec:
  selector:
    app: tomcat-laptop
  ports:
```

```
  - port: 8080
    targetPort: 8080
```

3. **vim ingress.yaml**

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: a42a79564f9b24090a15335a891c39f1-2015223809.ap-southeast-
1.elb.amazonaws.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-svc-home
            port:
              number: 80
      - path: /mobile
        pathType: Prefix
        backend:
          service:
            name: my-svc-mobile
            port:
              number: 80
      - path: /laptop
        pathType: Prefix
        backend:
          service:
            name: my-svc-laptop
            port:
              number: 8080
```

4. INSTALLING INGRESS-NGINX-CONTROLLER ON EKS

1. Install helm

1. curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
2. helm version

2. Install nginx-ingress using helm

1. helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
2. helm repo update
3. kubectl create namespace ingress-nginx
4. helm install ingress-nginx ingress-nginx/ingress-nginx \
   --namespace ingress-nginx \
   --set controller.service.type=LoadBalancer \
   --set controller.service.annotations."service\.beta\.kubernetes\.io/aws-load-balancer-type"="nlb" \
   --set controller.replicaCount=2

3. verify

1. kubectl get pods -n ingress-nginx
2. kubectl get svc -n ingress-nginx

5. kubectl apply -f deployment.yaml
6. kubectl apply -f service.yaml
7. kubectl apply -f ingress.yaml

8. kubectl get pods
9. kubectl get svc
10. kubectl get ingress

URL TO HIT ON WEB

1. <load_balancer_dns_name> ----------> nginx-home page
2. <load_balancer_dns_name>/mobile ----------> httpd-mobile page
3. <load_balancer_dns_name>/laptop ----------> tomcat-laptop page

---

IF U WANT TO DEPLOY INGRESS WITH HPA

Project structure:

```
ingress
|
|-- ingress.yaml
|
|-- httpd-deployment
|   |-- deployment.yaml
|   |-- hpa.yaml
|   |-- service.yaml
|
|-- nginx-deployment
|   |-- deployment.yaml
|   |-- hpa.yaml
|   |-- service.yaml
|
|-- tomcat-deployment
    |-- deployment.yaml
    |-- hpa.yaml
    |-- service.yaml
```

WRITING MANIFEST FOR THE KONG-INGRESS

1. vim deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-home
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-home
  template:
    metadata:
      labels:
        app: nginx-home
```

```
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd-mobile
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpd-mobile
  template:
    metadata:
      labels:
        app: httpd-mobile
    spec:
      containers:
      - name: httpd
        image: httpd:latest
        ports:
        - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tomcat-laptop
spec:
  replicas: 1
  selector:
    matchLabels:
      app: tomcat-laptop
  template:
    metadata:
      labels:
        app: tomcat-laptop
```

```
spec:
  containers:
  - name: tomcat
    image: tomcat:latest
    ports:
    - containerPort: 8080
```

2. **vim service.yaml**

```
apiVersion: v1
kind: Service
metadata:
  name: my-svc-home
spec:
  selector:
    app: nginx-home
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-svc-mobile
spec:
  selector:
    app: httpd-mobile
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-svc-laptop
spec:
  selector:
    app: tomcat-laptop
  ports:
  - port: 8080
    targetPort: 8080
```

3. **vim kong-ingress.yaml**

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kong-ingress
  annotations:
    konghq.com/strip-path: "true"
spec:
  ingressClassName: kong
  rules:
  - host: a42a79564f9b24090a15335a891c39f1-2015223809.ap-southeast-
1.elb.amazonaws.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-svc-home
            port:
              number: 80
      - path: /mobile
        pathType: Prefix
        backend:
          service:
            name: my-svc-mobile
            port:
              number: 80
      - path: /laptop
        pathType: Prefix
        backend:
          service:
            name: my-svc-laptop
            port:
              number: 8080
```

4. INSTALLING KONG-INGRESS-CONTROLLER

1. Install Helm

1. curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
2. helm version

2. Install kong-ingress

1. helm repo add kong https://charts.konghq.com
2. helm repo update
3. kubectl create namespace kong
4. helm install kong kong/kong \
   --namespace kong \
   --set ingressController.installCRDs=false \
   --set ingressController.enabled=true \
   --set proxy.type=LoadBalancer \
   --set proxy.annotations."service\.beta\.kubernetes\.io/aws-load-balancer-type"="nlb"

3. Verify kong pods

1. kubectl get pods -n kong
2. kubectl get svc -n kong

5. kubectl apply -f deployment.yaml
6. kubectl apply -f service.yaml
7. kubectl apply -f kong-ingress.yaml
8. kubectl get all -A
9. kubectl get ingress

URLS TO HIT IT ON WEB

1. <load_balancer_dns_name> ----------> nginx-home page
2. <load_balancer_dns_name>/mobile ----------> httpd-mobile page
3. <load_balancer_dns_name>/laptop ----------> tomcat-laptop page

Note:

1. you get the host_url when run kubectl get svc -n kong u have to copy these url and paste it in the ingress.yaml file on host section

---

WRITING MANIFEST FOR THE ISTIO-INGRESS

1. Install Istio CLI

1. curl -L https://istio.io/downloadIstio | sh -
2. cd istio-*
3. export PATH=$PWD/bin:$PATH
4. istioctl version

2.Install Istio with Ingress Gateway

1. istioctl install --set profile=demo -y
2. kubectl get pods -n istio-system

3.Enable Sidecar Injection

1. kubectl label namespace default istio-injection=enabled
2. kubectl get ns --show-labels

4.vim deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-home
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-home
  template:
    metadata:
      labels:
        app: nginx-home
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: httpd-mobile
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpd-mobile
  template:
    metadata:
      labels:
        app: httpd-mobile
    spec:
      containers:
      - name: httpd
        image: httpd:latest
        ports:
        - containerPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tomcat-laptop
spec:
  replicas: 1
  selector:
    matchLabels:
      app: tomcat-laptop
  template:
    metadata:
      labels:
        app: tomcat-laptop
    spec:
      containers:
      - name: tomcat
        image: tomcat:latest
        ports:
        - containerPort: 8080
```

5.vim service.yml

```
apiVersion: v1
kind: Service
```

```
metadata:
  name: my-svc-home
spec:
  selector:
    app: nginx-home
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-svc-mobile
spec:
  selector:
    app: httpd-mobile
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: my-svc-laptop
spec:
  selector:
    app: tomcat-laptop
  ports:
  - port: 8080
    targetPort: 8080
```

6.kubectl apply -f deployment.yaml
7.kubectl apply -f service.yaml

8. Restart Pods (Inject Envoy Sidecar)

1. kubectl rollout restart deployment nginx-home
2. kubectl rollout restart deployment httpd-mobile
3. kubectl rollout restart deployment tomcat-laptop

9. verify sidecar

1. kubectl get pods

10.Get or Copy Istio LoadBalancer DNS

3. kubectl get svc istio-ingressgateway -n istio-system

11. vim gateway.yml

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: app-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "*"
```

12.kubectl apply -f gateway.yml

13. vim virtualservice.yaml

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: app-routing
spec:
  hosts:
  - "*"
  gateways:
  - app-gateway
  http:
  - match:
    - uri:
        prefix: /mobile
    rewrite:
      uri: /
    route:
```

```
  - destination:
      host: my-svc-mobile
      port:
        number: 80

 - match:
   - uri:
       prefix: /laptop
   rewrite:
     uri: /
   route:
   - destination:
       host: my-svc-laptop
       port:
         number: 8080

 - match:
   - uri:
       prefix: /
   route:
   - destination:
       host: my-svc-home
       port:
         number: 80
```

14.kubectl apply -f virtualservice.yml

15.Verify Istio Resources

1. kubectl get gateway
2. kubectl get virtualservice

URLS TO HIT ON THE WEB

1. <istio_load_balancer> ----------> nginx-home page
2. <istio_load_balancer>/mobile ----------> httpd-mobile page
3. <istio_load_balancer>/laptop ----------> tomcat-laptop page

---

INGRESS REPOSITORY LINK (NGINX/KONG/ISTIO)

https://github.com/mayurpandit25/manifests/tree/e6bccb741c3820063dbe1ada9d66d2358dd63a0f/Ingress

---